UEES
UNIVERSIDAD ESPIRITU SANTO

CineSpot

Diseño de una solución concurrente

SISTEMAS OPERATIVOS · P2 GRUPAL

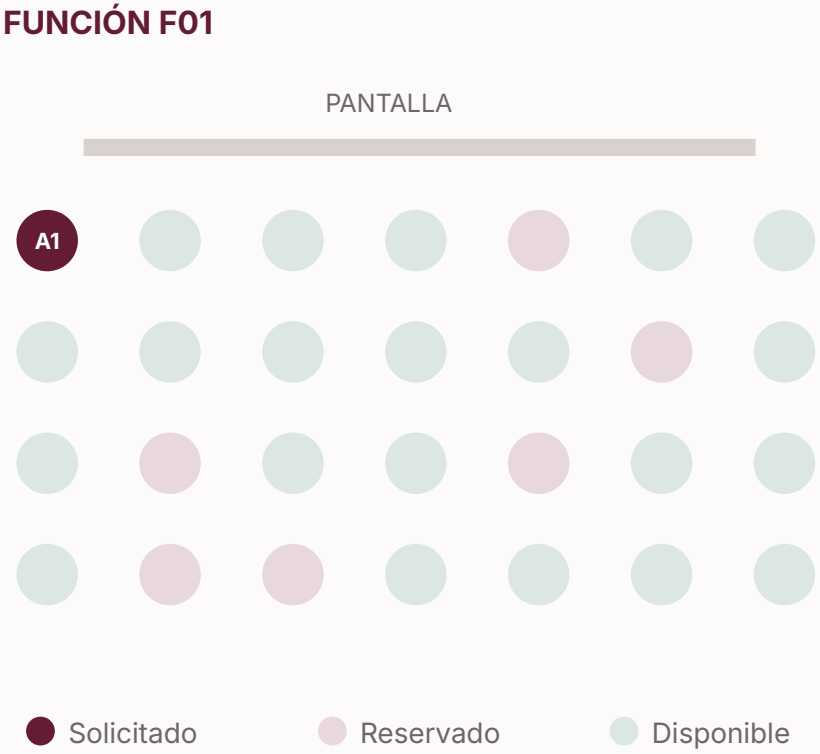
Josías Pérez · Jorge Colamarco

05 de noviembre de 2026

CineSpot: reservas bajo concurrencia

Simulación en Python de consulta y reserva de asientos para funciones de cine.

Varios usuarios pueden solicitar asientos para una misma función. Cada solicitud se atenderá desde un hilo de ejecución.



01 **Consultar disponibilidad**

02 **Enviar la solicitud**

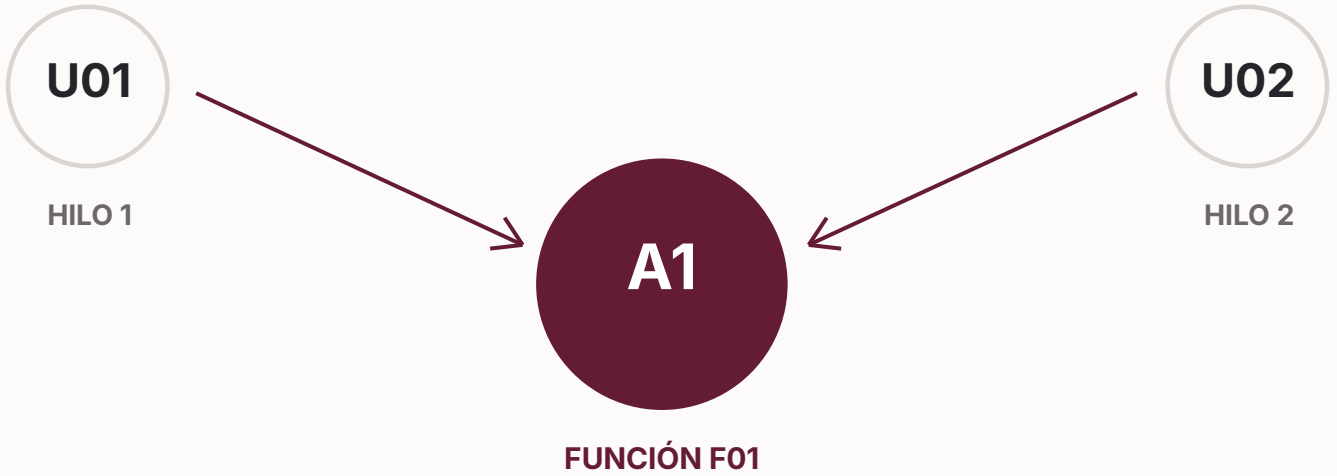
03 **Validar y confirmar**

EL RETO · Evitar dos confirmaciones para un mismo asiento.

Un recurso, múltiples solicitudes

El recurso compartido es el mapa de asientos de cada función.

Cada trabajador consulta y modifica el mismo estado.
El conflicto aparece si ambos leen “libre” antes de reservar.



QUÉ SE COMPARTE

Estado y usuario de cada asiento.

QUIÉN LO UTILIZA

Los hilos de reserva de una misma función.

DÓNDE SURGE EL RIESGO

En la lectura y escritura simultánea del estado.

Alternativa 1: un entero

Un atributo de tipo int expresa la disponibilidad de un asiento individual.

PYTHON FRAGMENTO DE DISEÑO

```python
class SalaCineBasica:
    def __init__(self):
        self._estado = 0
        self._lock = threading.Lock()

    def reservar(self):
        with self._lock:
            if self._estado == 1:
                return False
            self._estado = 1
            return True
```

0 **Disponible**

1 **Reservado**

LÍMITE DEL MODELO

No registra al usuario y representa un solo asiento.

Alternativa 2: un diccionario

Un mapa por función asocia cada asiento con su estado y usuario.

PYTHON FRAGMENTO DE DISEÑO

```python
class SalaCine:
    def __init__(self, id_funcion, ids_asientos):
        self.id_funcion = id_funcion
        self._asientos = {
            a: {"estado": 0, "usuario": None}
            for a in ids_asientos
        }
        self._lock = threading.Lock()
```

EJEMPLO · F01

ASIENTO	ESTADO	USUARIO
A1	Libre	—
A2	Reservado	U01

VENTAJA

Conserva información detallada y admite nuevos atributos.

Tres clases, funciones distintas

El diseño separa coordinación, solicitudes y acceso al recurso.

01

SimuladorCineSpot

Coordina la ejecución

ATRIBUTOS

funciones · solicitudes
trabajadores

MÉTODOS

ejecutar()

02

TrabajadorReserva

Atiende una solicitud

ATRIBUTOS

sala · id_asiento
usuario · resultado

MÉTODOS

run()

03

SalaCine

Protege el recurso

ATRIBUTOS

id_funcion · _asientos
_lock

MÉTODOS

consultar_asiento()
reservar_asiento()

Dos hilos, una misma instancia

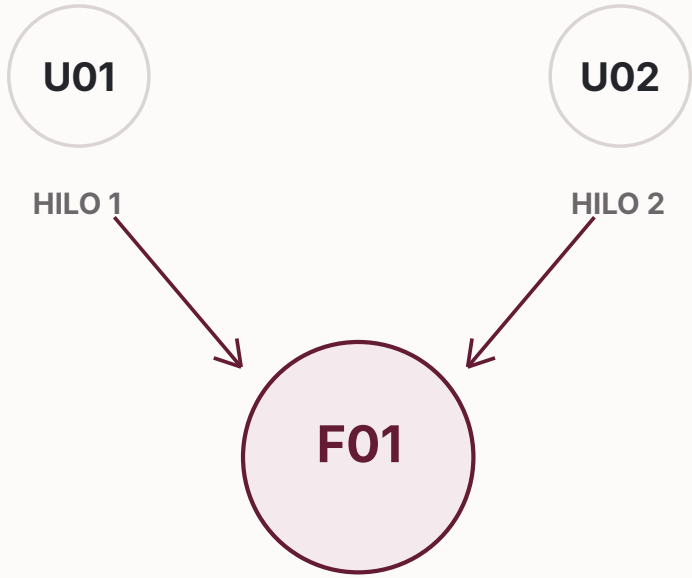
Ambas solicitudes reciben la misma referencia al objeto SalaCine de F01.

PYTHON FRAGMENTO DE DISEÑO

```python
sala = SalaCine("F01", ["A1", "A2"])

hilo1 = TrabajadorReserva(
    sala, "A1", "U01")
hilo2 = TrabajadorReserva(
    sala, "A1", "U02")

hilo1.start()
hilo2.start()
hilo1.join()
hilo2.join()
```



SalaCine: _asientos + _lock

UNA SOLA INSTANCIA

Así ocurre una condición de carrera

Ejemplo hipotético sin bloqueo: dos hilos intentan reservar A1 en F01.

PASO	HILO 1 / U01	HILO 2 / U02
01	Lee A1 como disponible	
02		Lee A1 como disponible
03	**Registra U01 y confirma**	
04		**Registra U02 y confirma**

RESULTADO INCORRECTO

Dos confirmaciones para un asiento; la segunda sobrescribe a la primera.

La sección crítica y el mutex

La consulta, validación y actualización se ejecutan bajo el mismo bloqueo.

PYTHON FRAGMENTO DE DISEÑO

```python
def reservar_asiento(self, id_asiento, usuario):
    with self._lock:
        registro = self._asientos.get(id_asiento)
        if registro is None:
            return "solicitud_invalida"
        if registro["estado"] == 1:
            return "asiento_ocupado"
        self._asientos[id_asiento] = {
            "estado": 1, "usuario": usuario
        }
        return "reserva_confirmada"
```

Adquirir

with self._lock

Validar y modificar

Leer el asiento y actualizar estado + usuario.

Liberar

Al salir del bloque with, incluso al retornar.

Elegimos el diccionario

Ambas alternativas necesitan sincronización, pero no conservan la misma información.

CRITERIO	ENTERO	DICCIONARIO
Disponibilidad de un asiento	Sí	Sí
Varios asientos	Requiere otra estructura	**Claves por asiento**
Usuario asociado	No directamente	**Sí**
Extensión del registro	Limitada	**Flexible**
Protección de reservas	Mutex	Mutex

DECISIÓN TÉCNICA

Un diccionario por función, protegido con un mutex.

REFERENCIAS

Fuentes académicas y documentación técnica empleadas en el diseño.

01 Pérez, J., y Colamarco, J. (2026, 13 de octubre). CineSpot: Análisis de modelos de comunicación. Universidad de Especialidades Espíritu Santo.

02 Pérez, J., y Colamarco, J. (2026, 29 de octubre). CineSpot: Planificación y diseño de la solución. Universidad de Especialidades Espíritu Santo.

03 Python Software Foundation. (s. f.). threading — Thread-based parallelism. Python 3.14 documentation. https://docs.python.org/3.14/library/threading.html

04 Python Software Foundation. (s. f.). Built-in types — Mapping types (dict). Python 3.14 documentation. https://docs.python.org/3.14/library/stdtypes.html#mapping-types-dict